Plateforme Android - pili-pos-android

Distributeur : Bruno Moyse / nuagemagique.dev (Bruxelles, BE) **Produit :** pili-pos-android 1.0.0 (posSwVersion pili-pos-android/1.0.0) **Plateforme :** Android 13 (API 33), terminal Sunmi D3 Pro (variante avec services Google) **Date :** À effectuer (EV-18 : date de dépôt) **Référence :** réponse aux §I.C, §II.C, §II.F et §II.G du manuel de certification pour la variante Android.

Le présent document décrit la plateforme sur laquelle tourne `pili-pos-android` : le matériel, l'application, le verrouillage du terminal, l'identifiant de l'appareil, le stockage, le réseau, l'absence d'accès à distance et le comportement en cas de panne. Il n'a pas d'équivalent dans le dossier Linux (`pili-pos` repose sur une image kiosque Debian, décrite dans les annexes Linux). Le moteur fiscal est le même code source sur les deux plateformes (voir `CADRE_DEMANDE_FR.md` §4).

Convention de lecture. Une section « tel que construit » décrit ce que le code de la version 1.0.0 fait. Une section « cible » décrit ce que le produit doit faire sur le terminal avec services Google ; elle était signalée par `EV-13` tant que la preuve n'était pas produite ; les mesures de l'image avec services Google sont faites (`evidence/gms/EV13_2026-10-15.md`), il reste ce qu'un vrai terminal seul établit (`EV-15`). Tout fait matériel qui ne peut être établi qu'avec un vrai terminal est signalé par `EV-15`. Les mesures disponibles à ce jour ont été faites sur un profil émulé du terminal (§1).

---

1. Matériel et système

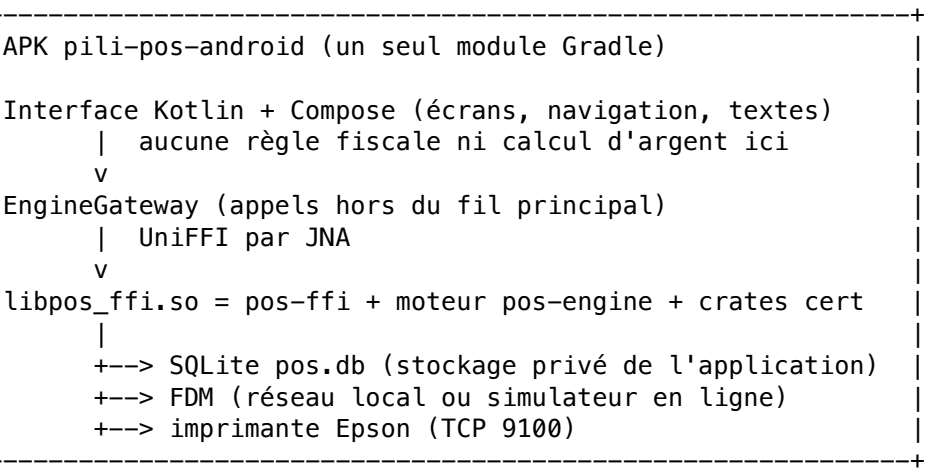
Élément	Valeur
Terminal	Sunmi D3 Pro, variante avec services Google (Google Play services et boutique Play présents sur l'image)
Système	Android 13 (API 33) ; l'application exige l'API 33 au minimum (`minSdk` 33)
Processeur	arm64-v8a uniquement : le seul fichier natif de l'APK est `libpos_ffi.so` pour cette architecture
Écran	1920 x 1080 en paysage (orientation capteur paysage imposée par l'application)
Imprimante	Epson TM-m30III (80 mm) sur le réseau local, TCP 9100 (ESC/POS), comme sous Linux
FDM	module2 : boîtier sur le réseau local en production, simulateur en ligne pour la machine de test (voir `SCHEMA_ASSEMBLAGE_FR.md` §7)
Application	un seul APK, identifiant `dev.nuagemagique.pilipos`, version 1.0.0, versionCode 10000

Le versionCode se calcule ainsi : MAJEUR x 10000 + MINEUR x 100 + CORRECTIF (1.0.0 donne 10000). Les versions de développement portent le suffixe `-dev` et ne sont jamais une version déclarée.

Machine de test. Sauf si un vrai terminal est disponible au moment du dépôt, la machine de test est le profil émulé `sunmi_d3_pro_gms_api33`, configuré avec : 1920 x 1080 à 160 dpi, 4 Go de mémoire, Android 13 arm64, orientation paysage. Ce sont les réglages du profil émulé, pas des faits établis sur le vrai terminal. Les mesures déjà faites (essais de verrouillage, de stockage, d'export, de mise à jour, de redémarrage) l'ont été sur le profil jumeau `sunmi_d3_pro_api33`, sans la boutique Play ; la répétition sur le profil avec services Google fait partie des preuves restant à établir (`EV-10`, `EV-13`).

Faits matériels du vrai terminal. Les caractéristiques réelles du D3 Pro (processeur, mémoire, stockage interne, emplacement de carte, ports physiques, démarrage vérifié, chiffrement du stockage, comportement de l'imprimante Epson réelle) ne sont pas établies par le dépôt : À effectuer (EV-15 : fiche mesurée d'un vrai D3 Pro, ou déclaration que la machine de test est le profil émulé).

2. Architecture de l'application



- **Un seul APK.** Interface en Kotlin et Compose, moteur fiscal en Rust. Le moteur est compilé en `libpos_ffi.so` (crate `android/rust/pos-ffi`, façade UniFFI au-dessus de `crates/pos-engine`) et l'interface l'appelle par UniFFI et JNA. Le même code source du moteur est utilisé sous Linux (application Tauri).
- **Aucune règle fiscale ni calcul d'argent en Kotlin.** L'interface affiche et transmet ; le moteur décide. Le calcul du rendu en espèces, de la ventilation de TVA et de l'aperçu de règlement passe par des fonctions d'aide du moteur (`docs/android/FFI.md` §9). Un contrôle automatique de l'arborescence interdit à la couche plateforme d'y toucher.
- **Le moteur possède les déclencheurs fiscaux.** L'événement P à la sortie d'une commande, l'enregistrement de l'entrée en service et de la sortie de service, le verrouillage après 5 minutes d'inactivité et le suivi du FDM sont des règles du moteur (`docs/android/ENGINE.md` §7). L'interface se limite à signaler l'activité de l'utilisateur et à réagir aux événements.
- **États du moteur.** Fermé (non provisionné, appareil différent, identifiant illisible, arrêt), en ouverture, prêt (la version affichée est pili-pos-android/1.0.0) ou fatal. Sans configuration provisionnée signée, le moteur n'est jamais démarré : il n'existe aucune identité de test implicite.
- **Service au premier plan.** Un service Android de type `specialUse` garde le moteur actif tant que la caisse est ouverte ; il ouvre le moteur au démarrage et le ferme à sa destruction.
- **Une seule activité.** `MainActivity` est l'écran d'accueil du terminal (kiosque, §3).
- **Journal fiscal.** Il est dans `pos.db`, géré par le moteur (voir `SECURITE_BD_FR.md`). Le journal d'audit de la plateforme (§3.8) n'est pas le journal fiscal.

---

3. Propriétaire de l'appareil et verrouillage

Le terminal est verrouillé par l'application elle-même, qui en est le **propriétaire de l'appareil** (device owner). Ce rôle est exigé dans toutes les versions : sans lui, l'application affiche un écran d'erreur et le moteur ne démarre pas.

3.1 Provisionnement sans compte Google

Le rôle de propriétaire ne peut être attribué que sur un appareil sans compte Google. La procédure est donc :

1. **Au banc du producteur** : sur un terminal neuf ou réinitialisé, sans aucun compte, l'APK est installé puis la commande `dpm set-device-owner` désigne l'application comme propriétaire. La commande retourne le code 0 même en cas de refus : le texte « Success » est le seul signal, il est vérifié.

2. **Sur le terrain, après une réinitialisation** : provisionnement par code QR (six appuis sur l'écran d'accueil de l'assistant de configuration) dont le contenu désigne l'APK et sa somme de contrôle SHA-256. Comportement sur l'image réelle de l'assistant de configuration : À effectuer (EV-15 : essai du provisionnement QR sur un vrai D3 Pro).
3. **Le producteur ne connecte jamais le terminal à un compte Google**, ni au banc, ni chez le client. Sur un appareil qui porte déjà un compte, la seule voie est la réinitialisation puis le provisionnement par code QR.
4. Après l'attribution du rôle, la restriction `DISALLOW_MODIFY_ACCOUNTS` (§3.3) interdit d'ajouter un compte.

Le refus d'attribution du rôle en présence d'un compte n'a pas été exercé sur l'émulateur (l'image n'en porte pas) : À effectuer (EV-15 : refus de `dpm set-device-owner` avec un compte présent).

La configuration de la caisse (identité fiscale, FDM, imprimante) arrive ensuite par un lot signé sur clé USB, vérifié par signature minisign avant tout usage (voir `MANUEL_INSTALLATION_FR.md`).

3.2 Mode kiosque (Lock Task)

À chaque démarrage de l'activité, le contrôleur de kiosque :

1. déclare l'application comme seul paquet autorisé en Lock Task ;
2. autorise d'abord le menu d'alimentation (fonction de Lock Task des actions globales) ; l'étape de politique `lock_task_features` (§3.4) fixe ensuite les fonctions de Lock Task à « aucune » à chaque création et reprise de l'activité : ni menu d'alimentation, ni bouton d'accueil, ni écran des applications récentes pour le personnel (le système remet la valeur par défaut à chaque redémarrage, d'où la réapplication) ;
3. fait de l'application l'écran d'accueil préféré et persistant ;
4. suspend les autres applications d'accueil et l'application Réglages du système (suspension réversible) ;
5. masque les barres système (mode immersif) et démarre le Lock Task.

Mesuré sur l'émulateur avec l'application prototype des essais préparatoires (mêmes appels de propriétaire d'appareil et de kiosque, pas pili-pos-android) : après un redémarrage ou un plantage, l'application est de nouveau au premier plan et verrouillée en 1 à 3 secondes. Le même essai avec pili-pos-android : À effectuer (EV-10 : redémarrage et verrouillage du produit dans le manuel d'exécution du D3 Pro). Le comportement avec le lanceur propre au terminal Sunmi et avec ses applications système n'est pas établi : À effectuer (EV-15 : lanceur Sunmi, extension kiosque Sunmi, listes de maintien en vie du fabricant).

3.3 Restrictions utilisateur (tel que construit)

Ensemble constant appliqué par `PolicySet.kt` (`android/app/src/main/kotlin/dev/nuagemagique/pilipos/platfo`
dans cet ordre, à chaque démarrage de l'application. Une restriction déjà présente n'est pas réappliquée.

Constante Android (`UserManager`)	Clé	Effet
`DISALLOW_FACTORY_RESET`	`no_factory_reset`	bloque la réinitialisation d'usine depuis les Réglages
`DISALLOW_SAFE_BOOT`	`no_safe_boot`	interdit le démarrage en mode sans échec (invite du menu d'alimentation)
`DISALLOW_UNINSTALL_APPS`	`no_uninstall_apps`	interdit la désinstallation d'applications
`DISALLOW_INSTALL_APPS`	`no_install_apps`	interdit toute installation d'application (boutique Play comprise) ; l'outil de mise à jour la lève pour sa seule session d'installation et la repose ensuite (§3.7)
`DISALLOW_INSTALL_UNKNOWN_SOURCES`	`no_install_unknown_sources`	interdit l'installation depuis des sources inconnues
`DISALLOW_INSTALL_UNKNOWN_SOURCES_GLOBALLY`	`no_install_unknown_sources_globally`	idem pour tous les profils

Constante Android (`UserManager`)	Clé	Effet
`DISALLOW_CONFIG_DATE_TIME`	`no_config_date_time`	verrouille la date, l'heure et le fuseau
`DISALLOW_ADD_USER`	`no_add_user`	interdit d'ajouter un utilisateur
`DISALLOW_USER_SWITCH`	`no_user_switch`	interdit de changer d'utilisateur
`DISALLOW_REMOVE_USER`	`no_remove_user`	interdit de supprimer un utilisateur
`DISALLOW_MODIFY_ACCOUNTS`	`no_modify_accounts`	interdit d'ajouter ou de modifier un compte (donc aucun compte Google)
`DISALLOW_USB_FILE_TRANSFER`	`no_usb_file_transfer`	interdit le transfert de fichiers par USB vers un ordinateur
`DISALLOW_NETWORK_RESET`	`no_network_reset`	interdit la réinitialisation des réglages réseau
`DISALLOW_SYSTEM_ERROR_DIALOGS`	`no_system_error_dialogs`	supprime les fenêtres de plantage ; un second plantage ne fige pas la caisse derrière une fenêtre
`DISALLOW_DEBUGGING_FEATURES`	`no_debugging_features`	coupe adb. **Versions de production uniquement**, appliquée en dernier et seulement quand un enregistrement de provisionnement existe ; les versions de développement ne l'appliquent jamais

Les quatorze premières lignes sont appliquées dans toutes les versions ; la dernière est propre aux versions de production.

Jamais appliquées (le code refuse de les appliquer et un test le vérifie) :

Constante Android	Raison
`DISALLOW_MOUNT_PHYSICAL_MEDIA` (clé `no_physical_media`)	la clé USB ou la carte SD doivent pouvoir se monter pour l'export vers le SPF ; le nom `no_mount_physical_media` est aussi interdit
`DISALLOW_CONFIG_WIFI` (clé `no_config_wifi`)	le Wi-Fi du site se prépare au banc avant l'import du lot ; un écran de réseau protégé par le code d'un Gérant est la cible, non construite en 1.0.0 (`MANUEL_INSTALLATION_FR.md` §4.3)

Mesuré sur l'émulateur avec l'application prototype des essais préparatoires (mêmes appels de propriétaire d'appareil) : l'écran de réinitialisation d'usine répond « Bloqué par votre administrateur », l'ajout d'un utilisateur est refusé, la désinstallation de l'application est refusée. Ces refus avec pili-pos-android : À effectuer (EV-10 : manuel d'exécution du D3 Pro). Mesuré sur le produit : les restrictions de l'ensemble constant sont présentes dans le système après le provisionnement. Le chemin de démarrage en mode sans échec par les touches de volume et le menu de récupération du fabricant ne sont pas établis : À effectuer (EV-15 : menu de récupération du D3 Pro, ce qu'il peut effacer ou installer sous ces restrictions).

3.4 Autres mesures du propriétaire (tel que construit)

Chaque étape s'exécute isolément : un refus est consigné et les suivantes continuent. Noms des étapes du code : `PolicyApplier.kt`.

Étape	Effet
`auto_time_zone_off`, `time_zone`, `auto_time_on`	fuseau automatique coupé, fuseau fixé sur Europe/Brussels (valeur écrite en dur, jamais le fuseau d'exécution), heure automatique (réseau) activée
`uninstall_block`	blocage de la désinstallation de l'application elle-même
`stay_awake`	écran allumé tant que le terminal est branché (`STAY_ON_WHILE_PLUGGED_IN`, valeur 7)
`keyguard_off`	écran de verrouillage du système désactivé (l'application possède son propre verrouillage par code PIN)
`status_bar_off`	barre d'état désactivée
`hide:<paquet>`	une étape par application Google de la liste du §3.7, masquée par `setApplicationHidden` ; un paquet absent de l'appareil ou déjà masqué est ignoré
`update_policy`	politique de mise à jour du système à fenêtre horaire de nuit, 03:00 à 05:00 (§3.6)
`lock_task_features`	fonctions de Lock Task à « aucune » (§3.2)

Quand l'application n'est pas propriétaire de l'appareil, rien n'est appliqué et l'état est consigné.

3.5 Débogage

Les versions de production coupent adb (restriction `DISALLOW_DEBUGGING_FEATURES`, dernière étape du provisionnement). Mesuré sur l'émulateur avec l'application prototype des essais préparatoires (même appel) : adb passe hors ligne 4 secondes après la commande. Avec pili-pos-android : À effectuer (EV-10 : provisionnement de production complet). Seule l'application pourrait le rétablir ; elle ne le fait pas en production et il n'existe aucune minuterie de réactivation. Le comportement d'adb sur une image `user` du fabricant (installation `adb install -d`, accès root, options pour développeurs) : À effectuer (EV-15 : adb sur l'image de série du D3 Pro).

3.6 Mise à jour du système Android

Tel que construit. Le propriétaire fixe une politique de mise à jour du système à fenêtre horaire (`DevicePolicies.kt`, méthode `setSystemUpdatePolicyWindowed`, étape `update_policy` de `PolicyApplier.kt`) : le système n'installe ses mises à jour que de 03:00 à 05:00, heure locale du terminal. Le fuseau étant verrouillé sur Europe/Brussels (§3.4), la fenêtre est celle de Bruxelles, en dehors des heures de service. Une politique de report ne repousserait une mise à jour que de trente jours, après quoi le système se mettrait à jour quand il l'entend, éventuellement pendant un service. Une mise à jour du système ne modifie jamais l'APK déclaré. Politique relevée sur le profil avec services Google le 15.10.2026 (EV-13, `evidence/gms/EV13_2026-10-15.md`) : type fenêtre horaire, début 03:00, fin 05:00, aucun gel. Le comportement de l'image du fabricant (mises à jour du système proposées par le fabricant, redémarrage dans la fenêtre) : À effectuer (EV-15 : mise à jour du système sur le D3 Pro réel).

3.7 Services Google (variante avec services Google)

Le terminal cible porte les services Google et la boutique Play. L'application n'est pas distribuée par la boutique : elle est installée par le producteur (§3.1) puis mise à jour par le seul outil de mise à jour déclarée (`GARANTIE_MAJ_LANGUES_FR.md` §2.1). Le tableau distingue ce que le code fait aujourd'hui de la cible.

Mesure	Tel que construit
Aucun compte Google	provisionnement sans compte (§3.1) et `DISALLOW_MODIFY_ACCOUNTS`

Mesure	Tel que construit
Boutique Play et applications Google visibles par l'utilisateur	masquées par le propriétaire (`setApplicationHidden`, étapes `hide:<paquet>`) : `com.android.vending` (boutique Play), `com.android.chrome`, `com.google.android.googlequicksearchbox` (application Google et Assistant), `com.google.android.youtube`, `com.google.android.apps.youtube.music`, `com.google.android.videos`, `com.google.android.gm`, `com.google.android.apps.maps`, `com.google.android.apps.photos`, `com.google.android.apps.docs`, `com.google.android.apps.nbu.files`, `com.google.android.apps.tachyon`, `com.google.android.apps.messaging` et `com.google.android.calendar`. Un paquet absent de l'appareil est ignoré. Les services Google Play (`com.google.android.gms`, `com.google.android.gsf`), l'interface système, les Réglages, l'installateur de paquets, le WebView, le clavier et la synthèse vocale ne sont jamais masqués (le code refuse de les masquer et un test le vérifie). Le mode kiosque suspend en plus les autres écrans d'accueil et les Réglages. Mesuré sur le profil avec services Google le 15.10.2026 (EV-13, `evidence/gms/EV13_2026-10-15.md`) : onze de ces paquets sont présents et masqués ; `com.google.android.videos`, `com.google.android.apps.nbu.files` et `com.google.android.apps.tachyon` sont absents de l'image ; la boutique Play et Chrome sont absents de `pm list packages` et présents avec `-u`. Le fournisseur de WebView est `com.google.android.webview` (Chrome masqué). Sur le terminal réel : À effectuer (EV-15 : liste des paquets du D3 Pro réel)
Installations	`DISALLOW_INSTALL_APPS`, `DISALLOW_INSTALL_UNKNOWN_SOURCES` et `DISALLOW_INSTALL_UNKNOWN_SOURCES_GLOBALLY` sont posées. L'outil de mise à jour lève `DISALLOW_INSTALL_APPS` pour sa seule session d'installation (la session du propriétaire est bloquée par cette restriction sur certaines versions d'Android) et la repose dès que le système a répondu ou que l'attente de 90 secondes est écoulée, quoi qu'il arrive ; si l'installation réussit, le processus est remplacé et le démarrage suivant la repose, comme après un plantage. Mesuré le 15.10.2026 (EV-13) : la mise à jour déclarée 1.0.0 vers 1.0.1 s'installe sous `DISALLOW_INSTALL_APPS` (parcours `plat_e2e.sh`, contrôle `update`) ; les 14 restrictions du code sont présentes.

Mesure	Tel que construit
Politique de mise à jour du système	fenêtre horaire de 03:00 à 05:00 (§3.6) ; relevée sur l'image le 15.10.2026 : fenêtre de 180 à 300 minutes, aucun gel (EV-13)
Play Protect	non piloté par l'application ; sur le profil émulé (sans compte Google), aucun message de Play Protect n'est apparu à l'installation ni à la mise à jour d'un APK installé hors boutique (EV-13) ; sur un vrai terminal : À effectuer (EV-15 : Play Protect sur le D3 Pro réel)

Si Play Protect bloque ou avertit, la parade est l'enregistrement du paquet et de la clé de signature auprès de Google, démarche du producteur.

Vérification des développeurs Android. Google prévoit d'exiger, sur les appareils certifiés, que les applications proviennent d'un développeur vérifié, pays par pays à partir de 2026. L'inscription de l'identité du producteur, du nom de paquet et de la clé de signature de production relève de la seule responsabilité du producteur, en dehors du produit certifié, et ne change rien au comportement fiscal. Aucune inscription n'est faite à ce jour.

Non concernés par la variante avec services Google : le comportement fiscal certifié, la somme SHA-1 reproductible de l'APK, l'outil de mise à jour déclarée, l'identifiant de l'appareil (§4) et l'imprimante Epson sur le réseau local.

3.8 Journal d'audit de la plateforme

Un fichier d'audit au format JSON Lines, privé à l'application, consigne les événements de la plateforme (politiques, export, mise à jour, surveillance) avec l'horodatage au fuseau de Bruxelles, le type, l'événement, le détail, la version et le processus. Chaque ligne est écrite puis synchronisée sur le disque ; le fichier tourne à 1 Mio sur 5 fichiers. Ce n'est pas le journal fiscal, il n'est envoyé nulle part et il fait partie de l'export vers le SPF (`SECURITE_BD_FR.md` §8).

3.9 Écran « À propos »

L'écran « À propos », visible de tous les utilisateurs, affiche la version, le code de version, la référence du moteur, les sommes SHA-1 et SHA-256 de l'APK, la somme SHA-1 de `libpos_ffi.so`, la somme SHA-256 du certificat de signature, l'identifiant de l'appareil, l'identifiant de caisse et de terminal, l'espace libre et la taille des sauvegardes. Il sert à comparer le terminal aux valeurs du dossier (`GARANTIE_MAJ_LANGUES_FR.md` §1.2).

---

4. Identifiant de l'appareil

L'identifiant de l'appareil (`deviceId`) est le **numéro de série matériel du terminal Sunmi**, lu par l'application en tant que propriétaire de l'appareil (réponse b de la FAQ producteurs, question 505 : « D'autres types d'identification sont possibles via un identifiant hardware unique (ex: numéro de série). »). La réponse a de la même question indique qu'« en général, l'identification se fait via l'adresse MAC propre de chaque device » ; la réponse b admet donc le numéro de série.

Règle	Détail
Source	`Build.getSerial()`, autorisé pour le propriétaire de l'appareil avec la permission de lecture de l'état du téléphone, accordée par le propriétaire lui-même

Règle	Détail
Lecture	au provisionnement, puis **à chaque ouverture du moteur** ; jamais une copie mémorisée
Format	le numéro de série tel que le système le renvoie, sans espace en tête ni en queue, sans préfixe ; de 1 à 600 caractères (limite de la spécification)
Comparaison	l'enregistrement de provisionnement conserve le numéro du jour de l'import ; un numéro différent bloque le moteur (état « appareil différent ») et empêche toute signature
Sortie du blocage	uniquement un nouvel import signé, qui enregistre le nouveau numéro et est consigné ; cas visé : une sauvegarde restaurée sur un autre matériel
Numéro illisible	vide, « unknown », plus de 600 caractères ou refus de sécurité : le moteur reste fermé, le terminal ne démarre pas
Valeurs exclues	aucun identifiant généré, aléatoire ou issu de `ANDROID_ID` (cette valeur dépend de la clé de signature et de l'utilisateur et change après une réinitialisation d'usine, ce n'est pas un identifiant matériel au sens de la question 505), aucune adresse MAC (masquée aux applications ordinaires), aucune propriété système (vide dans le processus de l'application)

Le numéro est affiché dans l'écran « À propos », écrit dans le fichier d'identité de l'export vers le SPF et imprimé par le moteur sur les tickets, comme sous Linux où il s'agit de l'adresse MAC.

Valeur de l'émulateur. `EMULATOR37X2X12X0` (17 caractères), lue avec l'application prototype des essais préparatoires, stable après redémarrage, avec ou sans Wi-Fi. **Format du vrai terminal** et confirmation que son micrologiciel renvoie le numéro au propriétaire : À effectuer (EV-15 : numéro de série réel et sa lecture par `Build.getSerial()`). **Valeur de la machine de test** telle qu'imprimée sur ses tickets : `EMULATOR37X2X12X0`, relevée sur les tickets de `TICKETS_EXEMPLES_FR.md` avec l'application 1.0.0 (« Input device MAC ») et sur l'écran « À propos » (capture A82).

---

5. Stockage des données et carte SD

5.1 Où se trouve la base SQLite : jamais sur la carte SD

La base de données du moteur est `pos.db`, dans le dossier `files/engine` du stockage interne **privé de l'application**. À côté d'elle se trouvent `files/engine/backups` (les sauvegardes après Z), le lot de provisionnement et les journaux (`android/app/src/main/kotlin/dev/nuagemagique/pilipos/provisioning/ProvisioningP`
Ce choix est volontaire : un journal fiscal placé sur un support amovible pourrait être retiré, échangé, corrompu ou lu ailleurs.

- Le manifeste ne déclare pas `android:installLocation` et ne demande aucune permission de stockage ; Android ne peut donc jamais déplacer les données vers une carte adoptée comme stockage interne.
- La sauvegarde dans le nuage et le transfert vers un autre appareil sont désactivés (`allowBackup` à faux, règles d'extraction qui excluent tout : `android/app/src/main/res/xml/data_extraction_rules.xml`).
- La carte SD est un volume amovible comme une clé USB, atteint uniquement par une autorisation SAF : cible de l'export du gérant vers le SPF (`SECURITE_BD_FR.md` §8) et source d'une mise à jour déclarée depuis une clé.
- L'export nocturne (`EXPORT_NOCTURNE_FR.md`) garde ses copies locales dans `files/engine/exports` et les envoie par HTTPS ; il n'écrit jamais sur la carte.

5.2 Ce que la caisse conserve

Après chaque rapport Z, le moteur écrit une copie complète de `pos.db` dans `files/engine/backups` et **conserve les 7 copies les plus récentes**, en supprimant les plus anciennes (règle du produit). Le journal fiscal contenu dans `pos.db` n'est jamais supprimé. Le commit du moteur qui porte cette règle, à l'intérieur de l'étiquette de gel : À effectuer (EV-19 : commit de la conservation des 7 sauvegardes dans `linux-v1.0.0-rc2`).

La caisse conserve donc environ **8 fois la taille de la base** (la base et 7 sauvegardes). Aux taux mesurés (§5.3) : environ 4 Go après un an à 100 ventes par jour et environ 13 Go à 300 ventes par jour. L'écran « À propos » affiche l'espace interne libre et la taille des sauvegardes, avec un avertissement sous 2 Gio libres ou quand les sauvegardes dépassent la moitié de l'espace libre ; la plateforme ne supprime rien elle-même.

5.3 Taille de la base : mesurée

Mesure sur l'émulateur (essai de bout en bout de la plateforme : 50 ventes d'une ligne puis 10 ventes de 6 lignes réglées en espèces, puis le rapport Z, moteur réel et FDM simulé), taille logique de la base avec ses pages (`page_count` x `page_size`, en mode journal WAL) :

Vente	Croissance de la base
une ligne	6 471 octets par vente
six lignes	14 336 octets par vente

Une ligne de plus coûte environ 1,6 Ko (1 573 octets, mesuré, non décomposé). Une vraie vente a plusieurs lignes : le chiffre à six lignes sert au dimensionnement, celui d'une ligne est un plancher. La sauvegarde faite après le Z de cet essai pèse 778 240 octets. Ces mesures remplacent une estimation antérieure, non mesurée.

Projection annuelle (octets par vente x ventes par jour x 365) :

Ventes par jour	Vente d'une ligne	Vente de six lignes
100	environ 0,24 Go par an	environ 0,5 Go par an
300	environ 0,7 Go par an	environ 1,6 Go par an

5.4 Taille de la carte SD recommandée

La carte ne sert qu'aux exports et aux mises à jour. Un export vers le SPF copie `pos.db` et les 7 sauvegardes les plus récentes : il pèse environ 8 fois la base. Recommandation :

- une carte microSDXC de bonne qualité, à haute endurance (classe A1 ou U3) ;
- formatée en **exFAT** : le FAT32 refuse les fichiers de 4 Gio et plus, seuil que la base seule atteint après quelques années à fort volume ;
- **64 Go** pour la première année à environ 100 ventes par jour : un export pèse environ 3,9 Gio à la fin de la première année, ce qui laisse la place d'une quinzaine d'exports ;
- **128 Go** à environ 300 ventes par jour ou à partir de la deuxième année : un export pèse alors environ 13 Go à la fin de la première année.

L'application ne supprime jamais un dossier sur la carte : le gérant efface les anciens exports. La carte n'est jamais l'unique copie des données : l'export vers le SPF est une action du gérant et la copie hors caisse relève de l'export nocturne.

5.5 Ce qui n'est pas établi

La capacité de stockage interne libre du vrai D3 Pro, la plus grande carte qu'il accepte, le comportement de son emplacement et la tenue d'une vraie carte exFAT (le disque SD virtuel de l'émulateur est en vfat) : À effectuer (EV-15 : espace interne libre, plus grande carte acceptée, essai d'un export sur une vraie carte exFAT).

6. Réseau et connexions

Le terminal n'ouvre **aucun port en écoute**. Il n'initie que des connexions sortantes :

Destination	Protocole	Condition
FDM module2 (boîtier du réseau local, ou simulateur en ligne pour la machine de test)	GraphQL en HTTPS avec certificat client (mTLS) et jeton, comme sous Linux ; le moteur accepte aussi le HTTP clair comme sous Linux	toujours
Imprimante Epson TM-m30III	TCP 9100 (ESC/POS) sur le réseau local	toujours
HubRise (`manager.hubrise.com`, `api.hubrise.com`)	HTTPS, port 443	seulement si HubRise est activé (`COMMANDES_EN_LIGNE_FR.md`)
Service d'envoi du ticket numérique (`api.scaleway.com`)	HTTPS, port 443	seulement si le ticket numérique est configuré (`TICKET_NUMERIQUE_FR.md`)
Point de réception de l'export nocturne	HTTPS	seulement si la section d'export de la configuration est renseignée (`EXPORT_NOCTURNE_FR.md`)
Registre public des artefacts déclarés (GHCR)	HTTPS, tirage anonyme	seulement quand le gérant lance une mise à jour (`GARANTIE_MAJ_LANGUES_FR.md` §2.1)

Règles :

- **Racines TLS embarquées.** Le moteur ne s'appuie pas sur le magasin de certificats d'Android : les racines de confiance sont compilées dans l'APK (mode `BundledRoots`) ; un jeu périmé échoue en refusant la connexion. Renouveler les racines suppose un nouvel APK, donc une nouvelle version déclarée. Un proxy qui inspecte le TLS avec une racine privée n'est pas accepté.
- **FDM en mTLS.** Le client mTLS du FDM (certificat serveur à X.509 v1 de module2) est le même code que sous Linux ; il a été vérifié sur Android lors des essais préparatoires.
- **Texte clair.** La configuration de sécurité réseau de l'application interdit le trafic en clair côté Kotlin, sauf vers `127.0.0.1` et `localhost` (simulateurs de l'environnement de test, joints par redirection de port adb).
- **Aucun trafic permanent.** Il n'existe aucune session d'assistance, aucun canal permanent et aucun serveur : la mise à jour est lancée par le gérant, l'export nocturne est celui de `EXPORT_NOCTURNE_FR.md`.
- **Réseau local.** Le terminal rejoint le réseau par Wi-Fi (préparé au banc avant l'import du lot ; `DISALLOW_CONFIG_WIFI` n'est jamais appliquée, mais l'application 1.0.0 n'a pas d'écran de réseau, `MANUEL_INSTALLATION_FR.md` §4.3) ou par liaison filaire si le modèle en offre une. Ports physiques du vrai D3 Pro : À effectuer (EV-15 : connectivité filaire et Wi-Fi mesurées sur un vrai D3 Pro).

---

7. Absence d'accès à distance

La version 1.0 ne comporte **aucun outil d'assistance à distance** (aucun équivalent de RustDesk). Le moteur est ouvert sans programme de support : la commande de session d'assistance répond « non disponible » et l'interface n'en propose pas l'entrée. Aucun port en écoute, aucun canal permanent, adb coupé sur un terminal provisionné (§3.5).

Le support se fait **sur place** : écrans du gérant protégés par PIN, lot de provisionnement signé sur clé USB (configuration, jeton du FDM, certificats mTLS) et export SAF des journaux et des sauvegardes sur clé USB ou carte SD. Ajouter un jour une assistance à distance serait une nouvelle version Android et une modification du dossier.

---

8. Arrêt, panne et reprise

- **Reprise au démarrage.** Au démarrage du terminal, l'application, écran d'accueil persistant, démarre et se verrouille en Lock Task ; le service au premier plan rouvre le moteur. Un récepteur de démarrage consigne l'événement et lance l'activité principale en secours. Mesuré sur l'émulateur avec l'application prototype des essais préparatoires : 21 à 23 secondes jusqu'à la fin du démarrage du système, application verrouillée 1 à 3 secondes plus tard. Le même essai avec pili-pos-android : À effectuer (EV-10).
- **Chien de garde.** Si le moteur passe à l'état fatal (échec d'ouverture ou panique du code Rust), la plateforme consigne l'événement, attend 3 secondes (les journaux s'écrivent), puis met fin à son propre processus ; le système redémarre l'écran d'accueil persistant et le service. Budget : 3 redémarrages en 10 minutes, au-delà l'écran d'erreur reste affiché. Mesuré : environ 3,6 secondes entre le déclenchement et l'état « prêt » du nouveau processus, dont 3 secondes d'attente voulue, sans second import. Un moteur fermé plus de 60 secondes hors opération de plateforme est relancé.
- **Mise à jour.** Après l'installation d'une nouvelle version, un récepteur relance l'activité principale ; l'application est de nouveau verrouillée en moins de 2 secondes (mesuré).
- **Coupure de courant.** Mesuré avec l'application prototype des essais préparatoires (mêmes appels de propriétaire d'appareil et de kiosque, sa propre base d'essai, pas `pos.db`) : après un arrêt brutal de l'émulateur (processus tué), l'application est verrouillée 3 secondes après la fin du démarrage et la base d'essai est intacte (contrôle d'intégrité « ok »), sans perte de ses lignes d'essai validées (écriture synchrone complète). Ce résultat n'est pas celui du produit. L'essai avec pili-pos-android et `pos.db`, arrêt brutal pendant une transaction puis démarrage à froid : À effectuer (EV-12 : reprise après coupure de courant sur le profil avec services Google). Cet essai ne modélise pas le cache d'écriture d'une vraie mémoire flash : À effectuer (EV-15 : coupure réelle sur un vrai D3 Pro).
- **Marche continue.** Un essai de 30 minutes avec l'application prototype des essais préparatoires sur l'émulateur a passé tous ses critères (un seul processus, verrouillage maintenu, base d'essai intacte). Ce résultat n'est pas celui du produit. L'essai de 24 heures avec le service au premier plan : À effectuer (EV-11 : essai de 24 heures).
- **Écran de veille et alimentation.** L'écran reste allumé tant que le terminal est branché (§3.4). Le sommeil profond du processeur, le comportement thermique et la touche d'alimentation en Lock Task sur le vrai matériel : À effectuer (EV-15).

---

9. Limites connues et vérifications sur matériel réel

Les mesures de ce dossier viennent d'un profil émulé. Les points suivants ne peuvent être établis qu'avec un vrai terminal ou sur le profil avec services Google ; chacun est tenu pour non établi tant que la preuve n'est pas produite.

Point	Statut
Numéro de série réel et sa lecture par le propriétaire (§4)	À effectuer (EV-15 : numéro de série réel)
Provisionnement par code QR, refus en présence d'un compte (§3.1)	À effectuer (EV-15 : assistant de configuration du D3 Pro)
Lanceur et applications système Sunmi, extension kiosque, listes de maintien en vie (§3.2)	À effectuer (EV-15)
Menu de récupération, démarrage sans échec par les touches de volume (§3.3)	À effectuer (EV-15)
adb sur l'image de série (§3.5)	À effectuer (EV-15)
Applications Google masquées, installations bloquées sauf l'outil de mise à jour, politique à fenêtre horaire, Play Protect (§3.6, §3.7)	mesuré sur le profil émulé le 15.10.2026 (EV-13) ; Play Protect et mise à jour du système sur un vrai terminal : À effectuer (EV-15)
Démarrage vérifié (Verified Boot) et chiffrement du stockage de l'image réelle	À effectuer (EV-15 : état du démarrage vérifié et du chiffrement du D3 Pro)

Point	Statut
Clé USB réelle et carte SD réelle : FAT32 et exFAT, montage automatique, retrait pendant une écriture, débit	À effectuer (EV-15)
Mise à jour : comportement de l'installateur de paquets Sunmi, durée d'installation sur le processeur réel	À effectuer (EV-15)
Imprimante Epson réelle : latence de la lecture d'état DLE EOT, papier, capot, erreurs, ligne en chinois (la page de codes actuelle imprime « ? »)	À effectuer (EV-15)
Affichage : densité, hauteur utile avec les barres masquées, polices, confort tactile	À effectuer (EV-15)
Durée de hachage bcrypt des codes PIN sur le processeur réel (295 ms sur l'émulateur, mesuré avec l'application prototype des essais préparatoires)	À effectuer (EV-15)
Marche continue de 24 heures, coupure de courant réelle, durée de démarrage jusqu'au verrouillage, livraison de l'événement de fin de démarrage	À effectuer (EV-11, EV-12, EV-15)
Racines TLS embarquées sur l'image réelle	À effectuer (EV-15)
Essai réel contre le simulateur en ligne module2 (`CPOS0031234567`, terminal `POS-2-ANDROID`)	À effectuer (EV-14 : essai en direct, sur accord du producteur)